TEST PLAN

Hospital Management System (HMS)

1. Purpose and Testing Methodology

This strategy defines which testing types will be used for the Hospital Management System (HMS), how each type will be performed, and which tools will support the work. The team will follow an iterative, risk-based approach, prioritizing authentication and role permissions, patient data, appointments, admissions/discharge, billing, and workflows with high business or data-integrity impact.

Testing workflow

1. Review repository documentation, available UI workflows, API specification, user roles, and business rules.
2. Identify risks and design test scenarios, test data, expected results, and traceability to in-scope requirements.
3. Prepare the environment and verify that frontend, backend, and database are available.
4. Run manual functional and exploratory tests, API tests, automated UI regression tests, and performance tests.
5. Log defects with reproduction steps, expected/actual results, severity, and supporting evidence.
6. Retest fixes, run relevant regression tests, measure quality metrics, and prepare a test summary.

2. Test Types, Methods, and Tools

2.1 Manual Testing Strategy — What and How

Test type / technique	How the team will apply it to HMS	Tools / evidence
Functional testing	Check expected workflows for patient/doctor management, appointments, admissions/discharge, invoices and payments.	Test cases, browser, screenshots, results
Positive testing	Use valid data and permitted actions; verify successful results and saved data.	Browser, test data
Negative testing	Try missing fields, invalid formats/IDs, incorrect credentials, unauthorized actions, and invalid workflow transitions.	Browser, API responses, defect reports
Boundary Value Analysis	For fields with defined limits, test values at, below, and above boundaries. Derive limits from actual requirements.	Test cases / checklist
Equivalence Partitioning	Group inputs into valid and invalid classes and test representative values from each class.	Test design worksheet
RBAC testing	Log in as ADMIN, DOCTOR, and RECEPTIONIST; verify permitted actions and denied actions at UI and API levels.	Role accounts, browser, Postman
Exploratory testing	Run time-boxed sessions on realistic journeys; vary action order, navigation, data and recovery; record notes and defects.	Browser, session charter, screenshots
Regression testing	After fixes/changes, rerun affected tests and critical workflows.	Manual checklist and automation results
Usability checks	Check navigation, field labels, validation/error messages, and visible feedback.	Browser, screenshots

2.2 API Testing Strategy — What and How

Primary tools: Postman for request collections, environments, assertions, and repeatable execution; Newman (Postman CLI runner) for running the same collections from the command line and in CI; Swagger UI for exploring documented endpoints and schemas. API tests will check backend behavior independently of the UI.

API test type	Execution method
Positive API testing	Send valid payloads and verify status, response body, and business outcome.
Negative / validation testing	Omit required fields, send invalid values/formats, and use non-existing IDs; compare with API specification.
Authentication testing	Check public/protected endpoints, valid/invalid credentials, missing tokens, refresh and logout behavior.
Authorization / RBAC	Call endpoints as each role and verify permitted actions succeed and restricted actions are denied.
Data integrity	Create/update records through APIs and verify the returned values by reading them back through GET endpoints.
Workflow / state transitions	Test appointment booking/completion/cancellation; admission/discharge; invoice creation/payment.
Error handling	Check response structure and consistency with documented API behavior.
Repeatable collections	Organize requests by module; use environment variables for base URL and credentials; add assertions; run the collections with Newman (CLI).

2.3 Automated UI Testing Strategy — What and How

Primary automation tool: Selenium WebDriver (Java 17) with Page Object Model (POM) architecture and Cucumber BDD syntax. Maven supports build and dependency management. The framework focuses on stable, high-priority end-to-end and regression scenarios across core user roles. Automation will focus on stable, high-priority end-to-end and regression scenarios rather than every exploratory test.

Element	Planned approach
Framework / language	Selenium WebDriver + Java 17 + Page Object Model (POM) + Cucumber BDD (Gherkin syntax) + Maven.
Test design	Group tests by feature; use reusable helpers and Page Object Model or an equivalent maintainable structure.
Synchronization	Use explicit waits for dynamic elements; avoid fixed sleeps where possible.
Test data	Use controlled synthetic data and repeatable setup/cleanup where supported.
Reporting	Capture pass/fail results and useful failure evidence such as screenshots and logs.
Priority	Automate critical smoke/regression flows first; expand when UI and test data are stable.

2.4 Performance Testing Strategy — What and How

Primary tool: Apache JMeter for load generation, assertions, and reports (k6 is an acceptable alternative if the team prefers a script-based approach). Performance tests will be run against a dedicated test environment with synthetic data only, never against real patient data. The goal is to measure response time, throughput, error rate, and basic stability of the critical HMS flows under expected and elevated load. Acceptance thresholds, the number of concurrent users, and the test duration must be agreed by the team before execution.

Performance test type	How the team will apply it to HMS	Tool / evidence
Baseline (single-user)	Measure response time of each critical endpoint with one user to set a reference before applying load.	JMeter, Postman timings, summary report
Load testing	Simulate the expected number of concurrent users (for example receptionists, doctors, and admins) running realistic workflows for a fixed duration.	JMeter thread groups, aggregate report, response-time graphs
Stress testing	Gradually increase users beyond the expected load to find the breaking point and observe how the system fails and recovers.	JMeter ramp-up, error logs, application logs
Endurance (soak) testing	Run a moderate steady load for an extended period to detect slowdowns, memory growth, or connection-pool exhaustion.	JMeter, resource monitoring, application logs
Front-end page load check	Observe page load time and slow network requests for key screens (login, patient list, appointments, billing).	Browser DevTools (Network/Performance)

3. SMART Testing Objectives

The targets below are proposed and should be assigned dates in the team schedule. They are not actual test results.

ID	Objective / measurable target
OBJ-01	Execute 100% of approved in-scope functional test cases before test closure.
OBJ-02	Execute 100% of planned critical authentication/RBAC scenarios; have zero unresolved Critical security defects at closure.
OBJ-03	Execute 100% of planned API cases and aim for at least 95% API pass rate; investigate and document every failure.
OBJ-04	Validate data integrity for patient, doctor, appointment, admission, and invoice workflows using synthetic data.
OBJ-05	Automate 100% of high-priority regression scenarios agreed by the team by the agreed automation milestone.
OBJ-06	Complete agreed exploratory sessions and record every confirmed defect with reproducible evidence.
OBJ-07	Execute the agreed performance scenarios (baseline, load, and stress) and report response time, throughput, and error rate against the agreed thresholds; document every threshold breach.

4. Quality Metrics

Metric	Calculation / interpretation	Proposed target
Test execution progress	Executed test cases ÷ planned test cases × 100	100% executed or remaining tests formally dispositioned
Test pass rate	Passed ÷ executed × 100	≥ 95%; investigate every failure

Metric	Calculation / interpretation	Proposed target
Requirements coverage	In-scope requirements linked to tests ÷ total in-scope requirements × 100	100% mapped
Critical workflow coverage	Critical workflows tested ÷ identified critical workflows × 100	100%
API execution coverage	API cases executed ÷ planned API cases × 100	100% or approved exclusions documented
Automation coverage	Agreed high-priority regression scenarios automated ÷ agreed scenarios × 100	100% of agreed scope
Defect retest rate	Fixed defects retested ÷ defects ready for retest × 100	100%
Open Critical defects	Count of unresolved Critical defects	0 at closure
Open High defects	Count of unresolved High defects	0 unless formally accepted with rationale
Critical regression pass rate	Passed critical regression tests ÷ executed critical regression tests × 100	100% before closure
Performance threshold compliance	Performance scenarios meeting agreed thresholds ÷ performance scenarios executed × 100	100% of agreed scenarios, or breaches documented and accepted

A high pass rate alone does not prove readiness. Review coverage, defect severity, known risks, and environment limitations alongside the metrics.

5. Entry Criteria

Testing phase	Criteria to start
General	Scope/requirements reviewed; responsibilities agreed; test cases reviewed; defect-tracking process agreed.
Environment setup	Java/Maven, Node/npm, PostgreSQL and/or Docker setup available; frontend/backend start; database reachable.
Manual / exploratory	Usable build available; core navigation accessible; test data and expected behavior/assumptions documented.
API testing	API running; Swagger or endpoint documentation accessible; Postman environment and data prepared; Newman available.
UI automation	Browser and Selenium/Java project configured; target UI flows stable; test accounts and data available.
RBAC/security	Role accounts/tokens available; expected permissions and protected endpoints identified.
Regression/retest	Fix/build deployed; affected tests identified; basic smoke tests pass.
Performance testing	Functional smoke tests pass; dedicated test environment available; JMeter (or agreed tool) configured; scenarios, user counts, duration, and thresholds agreed; synthetic data prepared.

6. Exit Criteria

Testing phase	Criteria to finish
Manual / exploratory	Planned cases executed or dispositioned; critical workflows covered; confirmed defects logged; session findings documented.
API testing	Planned API cases executed or exceptions documented; response, validation, auth/RBAC, and data checks recorded; failures triaged.
UI automation	Agreed smoke/regression suite executed; results reviewed; failures classified as product, test, or environment issues.
Security / RBAC	Planned critical authentication and role-access scenarios completed; no unresolved Critical security defect at closure.
Performance testing	Agreed baseline, load, and stress scenarios executed; results compared with agreed thresholds; bottlenecks and failures documented and triaged; performance report reviewed.
Overall closure	Coverage and metrics reported; critical regression passes; no unresolved Critical defects; High defects resolved or formally accepted; risks/limitations documented; summary reviewed.

7. Suspension and Resumption Criteria

Suspend the relevant testing activity if:

- The application or critical service is unavailable, repeatedly crashes, or fails basic smoke checks.
- The database is unavailable or test data is corrupted, making results unreliable.
- A blocking authentication/RBAC defect prevents access to required roles or critical workflows.
- A major defect blocks patient registration, appointment handling, or another workflow needed for planned tests.

- Required credentials, environment variables, or dependencies are missing for the activity.
- The environment is too unstable to distinguish product defects from environment failures.

Resume when:

- The blocker is fixed or an agreed workaround is available.
- Application/database availability is confirmed and smoke checks pass.
- Required test accounts, test data, and configuration are restored.
- The team reviews affected tests and confirms the environment can produce reliable results.

8. Product Analysis

8.1 Product Overview

The Hospital Management System (HMS) is a full-stack web application for managing core hospital operations: users, patients, doctors, appointments, admissions/discharge, and billing. It also sends automatic WhatsApp notifications to patients when appointments are booked or cancelled.

8.2 Architecture and Technology

Layer	Technology
Frontend	React + Vite (served by nginx in Docker)
Backend	Java 17, Spring Boot 3 (Web, Data JPA, Bean Validation)
Security	Spring Security + JWT, Google OAuth login
Database	PostgreSQL
API Docs	SpringDoc OpenAPI (Swagger UI)
Build / Deploy	Maven, Docker Compose (postgres, backend, frontend)
External services	Google OAuth Provider, WhatsApp/Twilio API

High-level flow: Users interact with the React web interface. The frontend authenticates through username/password or Google OAuth, then sends REST requests with a JWT bearer token. The JWT filter authorizes each request before it reaches the business modules, which read and write to PostgreSQL.

8.3 Key Features (Modules)

1. **Authentication and User Management:** login, Google OAuth, refresh-token rotation, logout with token blacklisting.
2. **Patient Management:** create, view, and update patient records.
3. **Doctor Management:** doctor profiles linked to user accounts, with specialization.
4. **Appointment Scheduling:** book, cancel, and complete appointments.
5. **Admission and Discharge:** record admission and discharge dates and status.
6. **Billing and Invoices:** invoices per patient with amount and status.
7. **Notification System:** asynchronous WhatsApp notifications using the transactional outbox pattern.

8.4 Dependencies and External Systems

- **Google OAuth Provider** (login).
- **WhatsApp/Twilio API** (notifications, which can be mocked or simulated).
- **PostgreSQL** and the **Docker environment** (frontend :5173, backend :8080, Swagger UI, database :5433).

8.5 Stakeholder Questions

Answers are derived from the project documentation.

Question	Answer
Main objectives?	Digitize hospital workflows with secure, role-based access and reliable patient notifications.
Target audience?	Hospital staff: Admin, Receptionist, Doctor. The patient receives notifications and is not an RBAC role in the diagrams.

Question	Answer
Most important features?	Appointment booking with notifications, role-based access, patient and billing records.
Risks and challenges?	Notification reliability (retries, failures), token security, permission errors, data consistency in the booking transaction, external service availability.
Quality metrics?	To be agreed with stakeholders (for example: all critical defects closed, all RBAC rules verified, 100% of critical test cases passed).

9. Scope

9.1 In Scope

Area	What will be tested
Authentication	Username/password login, Google OAuth login, JWT validation, refresh-token rotation, logout and token revocation.
Authorization (RBAC)	Admin, Receptionist, and Doctor permissions for every API, including blocked and view-only access.
Patient Management	Create, read, update; input validation.
Doctor Management	Doctor profiles, specialization, user linkage.
Appointments	Book, cancel, complete; doctor/patient association and status changes.
Admission and Discharge	Admission creation, discharge date and status updates.
Billing and Invoices	Invoice creation, amounts, and status.
Notification System	Outbox record creation within the booking transaction, worker polling, SENT / PENDING / FAILED transitions, retry and backoff, simulation mode.
API	REST endpoints (via Swagger and an API client), status codes, error handling, X-Correlation-Id.
Frontend (UI)	Main user flows for each role, form validation, navigation, browser compatibility.
Performance	Baseline, load, stress, and endurance behavior of critical HMS endpoints and key UI screens (response time, throughput, error rate) using JMeter.
Deployment	Docker Compose startup and communication between frontend, backend, and database.

9.2 Out of Scope

Suggested items; confirm with the team.

- Real WhatsApp delivery in production (testing uses simulation mode or a mock/sandbox).
- Internal behavior of Google OAuth and Twilio, since they are third-party systems.
- A patient login portal or mobile app (not shown in the architecture).
- Hospital-specific compliance and regulatory certification.
- Database testing (schema and constraints, referential integrity, direct SQL verification of stored data). PostgreSQL is used only as part of the test environment.

10. Test Deliverables

The QA team will prepare and maintain the following deliverables to document, track, and evaluate the testing activities

of the Hospital Website.

The testing scope includes Manual Testing, Automation Testing, Functional Testing, UI Testing, Usability Testing, Compatibility Testing, Regression Testing, Performance Testing, and API Testing.

10.1 Testing Deliverables

Deliverable	Description
Manual Test Cases	Document test scenarios, test steps, test data, expected results, and actual results for the hospital website features.
Automation Test Scripts	Prepare automated test scripts for selected repetitive and critical test cases, such as login, registration, and appointment booking, where available.
API Test Cases and Results	Document API requests, responses, status codes, validations, and error-handling test results, where API access is available.
Bug Reports	Record identified defects with descriptions, steps to reproduce, expected results, actual results, severity, priority, and supporting screenshots. Defects are logged and tracked in Jira Software.
Test Execution Report	Track executed test cases and their results, including Passed, Failed, Blocked, and Not Executed.
Automation Execution Report	Document automated test execution results, including passed and failed automated tests.
Performance Test Results	Document the results of approved performance checks, including identified issues and observations.
Test Summary Report	Provide a final overview of testing activities, test coverage, defects, execution results, limitations, and overall testing status.

All test cases, bug reports, and testing results will be organized and tracked using **Jira Software with Xray** as the primary tool for bug and test case management. **Microsoft Excel or Google Sheets** can be used as an alternative/backup and for supplementary sheets, such as execution summaries and reports.

10.2 Deliverables Management

- Testing documents will be updated regularly throughout the testing process.
- Each defect will be recorded with sufficient information to support investigation and retesting.
- Automation scripts and API test collections will be maintained when these testing activities are implemented.
- Final testing documents and reports will be reviewed by the QA team before project submission.

Scope: Test Environment and Dependencies | Version 1.0 | October 25, 2026

11. Dependencies

11.1 System Dependencies

Dependency	Description	Impact if unavailable
PostgreSQL 16	Stores users, patients, doctors, appointments, admissions, invoices, revoked tokens, and notification_outbox	All functional testing blocked
Spring Boot backend	REST API, security, scheduling	Frontend and API tests blocked
Frontend (React + nginx)	UI layer and API proxy	UI/E2E tests blocked

Dependency	Description	Impact if unavailable
Maven and npm registries	Dependency downloads for builds	Build failures
Apache JMeter	Load generation, assertions, and performance reports	Performance tests blocked
Docker images	Base images for postgres, JDK, nginx	Environment cannot be provisioned
Scheduled jobs	Revoked-token cleanup and the outbox delivery worker	Token cleanup and notification retry tests blocked
Logging	SECURITY_AUDIT logger and X-Correlation-Id header	Audit and traceability checks blocked

11.2 External Dependencies

Service	Used for	Test approach
Google OAuth 2.0	Social login (/oauth2/authorization/google)	Real Google test project and test user; mock for automated runs
WhatsApp provider (generic webhook)	Appointment booking and cancellation notifications	Mock HTTP endpoint (WireMock or similar) to verify payloads, retries, and failures
WhatsApp provider (Twilio)	Same, via form-post mode	Twilio sandbox or test credentials
Simulation mode	Default notification mode: messages are logged, not sent	Used for most functional testing

Contingency plan: use mock services for the WhatsApp provider and Google OAuth if live sandboxes are unavailable.

11.3 Database Requirements

- DBMS: PostgreSQL 16.
- Isolation: a dedicated test database (default name hms_db) on localhost.
- Schema: must match the latest backend entities and migrations.
- Data volume: at least 10,000 patients and 50,000 appointments and invoices for performance testing.
- Backup/restore: database dump before each cycle for fast reset.

12. Test Environment

12.1 Environment

Environment	Purpose	Configuration
Local (localhost)	Functional, integration, regression, API, and performance testing	Docker Compose full stack (postgres, backend, frontend via nginx), or mvn spring-boot:run + npm run dev with a local PostgreSQL

12.2 Hardware

Component	Minimum	Recommended
Test machine (runs frontend, backend, and database)	Intel i5 / equivalent, 8 GB RAM, 30 GB free disk	Intel i7 / equivalent, 16 GB RAM, 50 GB SSD

12.3 Software

Layer	Specification
Backend runtime	Java 17 (JDK), Maven, Spring Boot 3
Frontend runtime	Node.js and npm (LTS version supported by Vite), React 18
Database	PostgreSQL 16
Containerization	Docker and Docker Compose (nginx serves the frontend and proxies /api, /oauth2, /login to the backend)
API documentation	Swagger UI at /swagger-ui/index.html, OpenAPI JSON at /v3/api-docs
Version control	Git / GitHub

The testing tools used for each test type are listed in **Resource Planning – Software, Hardware, and Technical Tools**.

12.4 Operating Systems

Windows 10/11 (with Docker Desktop) or Ubuntu 22.04 LTS.

12.5 Browsers

Browser	Version
Google Chrome	Latest
Microsoft Edge	Latest
Mozilla Firefox	Latest

Resolution: 1920×1080 and 1366×768 desktop only.

12.6 Network Configuration

Item	Detail
Frontend	http://localhost:5173
Backend API	http://localhost:8080
PostgreSQL	localhost:5432 (local run) or localhost:5433 (Docker Compose)
Swagger	http://localhost:8080/swagger-ui/index.html
OAuth redirect URI	http://localhost:8080/login/oauth2/code/google registered in Google Cloud Console
Connectivity	Internet needed only for Google OAuth and live WhatsApp provider tests; all other tests run offline

12.7 Test Configuration (Environment Variables)

Variable group	Test setting
Database	DB_URL, DB_USERNAME, DB_PASSWORD pointing to a dedicated test database
JWT	JWT_SECRET (replace the default), JWT_ACCESS_EXPIRATION_MS=900000 (15 min), JWT_REFRESH_EXPIRATION_MS=604800000 (7 days). Lower the expirations to test token expiry quickly

Variable group	Test setting
OAuth	GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, APP_FRONTEND_URL=http://localhost:5173
Notifications	NOTIFICATION_SIMULATION_ENABLED=true for most cycles; false for dedicated provider tests
Outbox tuning	NOTIFICATION_OUTBOX_POLL_MS, _BATCH_SIZE, _MAX_ATTEMPTS, _BASE_RETRY_SECONDS, and HTTP timeouts (reduced to speed up retry testing)

12.8 Test Data and Accounts

- Seeded accounts: admin (ADMIN), doctor (DOCTOR), reception (RECEPTIONIST), used for role-based access testing.
- Synthetic data only: patients, doctors, appointments, admissions, and invoices. No real patient data is used.
- Reset strategy: docker compose down -v plus restart to return to a clean baseline before each cycle.
- Google test account for OAuth login scenarios.

13. Risk Management

This section lists the risks that could affect testing of the Hospital Management System, explains how we plan to reduce each one, and shows how the modules depend on each other, so we know what to adjust if something slips. The risks are based on the system's architecture and flow diagrams: a React front end, a Spring Boot 3 backend with JWT security, Google OAuth login, a PostgreSQL database, and WhatsApp notifications sent through a transactional outbox.

13.1 Identified Risks

Likelihood and impact are rated Low, Medium or High. Anything rated High on both, or High on impact, gets attention first.

ID	Risk	Likelihood	Impact	Mitigation Strategy
R1	Google **OAuth** login is unavailable, or the redirect URI / client credentials are misconfigured in the test environment	Medium	High	Use **pre-generated JWT tokens** for each role so API and RBAC tests don't depend on Google. Keep a small set of manual OAuth checks for when the provider is reachable.
R2	WhatsApp / Twilio API is slow, rate-limited, or not available (sandbox limits, expired credentials)	High	Medium	Replace the external call with a mock server (e.g. WireMock) that can return 200, timeout and error responses on demand. Run one real end-to-end check near the end if credentials allow.
R3	Notification outbox behaves unpredictably: the 5-second poller causes flaky tests, duplicate messages, or messages stuck in PENDING	Medium	High	Test the outbox by checking application logs, mock-provider requests, and status changes instead of waiting on fixed sleeps. Cover the failure path explicitly (attempts counter, PENDING to FAILED) and confirm an appointment is still saved when WhatsApp is down.
R4	Role permissions are wrong, e.g. a Receptionist editing doctor data or a Doctor changing billing records	Medium	High	Build a role-vs-endpoint matrix from the RBAC diagram and test every cell, including negative cases that should return 403. Re-run the full matrix after any security config change.
R5	Double booking: two receptionists book the same doctor at the same time slot	Medium	High	Add concurrency tests on POST /api/appointments and check how the system rejects the second request. Raise any gap as a defect straight away because it affects patients directly.

ID	Risk	Likelihood	Impact	Mitigation Strategy
R6	Test database differs from production-like setup, or tests leave data behind and affect each other	Medium	Medium	Use a dedicated PostgreSQL test database with seed and reset scripts. Every test creates its own data and does not rely on records from another test.
R7	Real patient data ends up in the test environment	Low	High	Only synthetic data (fake names, phone numbers and emails) is allowed. Nobody copies production data into testing, and the team agrees on this before testing starts.
R8	Missing or late deliverables: unfinished modules (e.g. Billing or Admission), incomplete Swagger docs, or no stable build to test	High	High	Agree early on the minimum build required to start each test phase. Test finished modules first, stub the missing ones, and ask developers to flag API changes so test cases can be updated quickly.
R9	Requirements or flows change after test cases are written (e.g. new statuses for appointments or invoices)	Medium	Medium	Keep test cases linked to the flow diagrams, so changes are easy to trace. Review the affected cases whenever a diagram or API changes.
R10	Team member leaves or falls ill during testing	Medium	Medium	Every module has a main and a backup tester. Test cases and results are kept in a shared place so anyone can pick up where someone stopped.
R11	Not enough time to run every planned test (shortened schedule, delayed builds)	High	Medium	Prioritise by risk: login, appointment booking, notifications and billing accuracy come first. Low-risk UI and cosmetic checks move to the end and can be dropped first if needed.
R12	System is slow or fails when many users book appointments or load patient lists at once	Low	High	Run basic load tests with JMeter or a similar tool on the appointment, login and patient search endpoints, and record response times and error rates.
R13	Billing calculation errors (rounding, wrong invoice status, wrong amount after discharge)	Low	High	Check invoice amounts against hand-calculated expected values, including boundary cases like zero and very large amounts, and test the invoice status changes.

13.2 Dependency Mapping

Not all modules can be tested independently. Appointments need patients and doctors, notifications need appointments, and invoices need admissions. If one piece is late, everything downstream is affected. The table below shows what each area relies on and what we will do if that dependency is not ready.

Module / Area	Depends On	What Goes Wrong If It Slips	Planned Adjustment
Authentication (Google OAuth + JWT)	Google OAuth provider, user and role seed data	Almost every other test needs a valid token	Use pre-generated JWTs per role so other testing can continue while login is fixed.
Patient and Doctor management	Authentication, database seed data	Appointments, admissions and invoices all need existing patients and doctors	Load patients and doctors directly into the test DB if the screens or APIs are delayed.
Appointment booking	Auth, Patient, Doctor modules	Notification tests cannot start until booking works	Insert appointment and outbox rows manually to test the notification flow in parallel.

Module / Area	Depends On	What Goes Wrong If It Slips	Planned Adjustment
Notification outbox and worker	Appointment booking, PostgreSQL, WhatsApp / Twilio API	Delayed or failed messages, duplicate sends	Mock the WhatsApp API and test the poller on its own with prepared PENDING records.
Admission and discharge	Patient module	Billing totals depend on admission dates	Use fixed admission and discharge dates in test data until the module is stable.
Billing and invoices	Patient and Admission modules	Wrong invoices if upstream data or logic is wrong	Test calculation rules in isolation first, then retest with real admission data.
RBAC checks	Authentication, all API modules	Gaps stay hidden if endpoints are added late	Update the role matrix every time a new endpoint appears and rerun it.
Performance testing	Stable functional build, prepared test data volume, isolated test environment	Results become unreliable if functional blockers or environment issues remain	Run baseline checks on a smoke-passing build first; fix or log blockers before load and stress runs.

In short, the main chain is: Authentication → Patient and Doctor data → Appointment booking → Notification outbox → WhatsApp. A second chain runs from Patient data → Admission → Billing. Delays on the first link of either chain hold up everything after it, so these get the earliest testing attention and the most backup options.

13.3 Monitoring and Contingency

- **Weekly risk review:** at each team check-in we go through the list, update likelihood and impact, and add anything new.
- **Ownership:** each risk has one team member responsible for tracking it, so nothing is left to "someone" to handle.
- **Early warning:** if a build is more than two days late or a module fails its basic smoke test, we tell the team lead immediately and switch to the adjustments in the dependency table.
- **Fallback plan:** if time runs out, we finish all high-priority tests (login, RBAC, appointments, notifications, billing) and record clearly which lower-priority tests were not run and why.
- **Defect handling:** a defect that blocks other tests is logged as critical and reported the same day.

14. Testing Schedule and Milestones

14.1 Overview & Timeline Strategy

The testing schedule follows an Agile/Iterative model from October 2026 through December 10, 2026, with initial preparation beginning in September 2026. All six team members participate across the testing phases (Manual, API, Automation, and Performance) to support shared learning and execution. Phase durations vary from 1 to 4 weeks depending on testing complexity and deliverables; the schedule includes defined handoffs and submission checkpoints.

Milestone
10 Days: Manual Testing Milestone
7 Days: Sprint 1 - API Testing

Milestone
14 Days: Sprint 2 - Web UI Automation (Selenium WebDriver + POM + Cucumber BDD)
7 Days: Sprint 3 - Performance Testing (JMeter) & API Collection (Newman)
3 Days: Sprint 4 - CI/CD Integration (Jenkins, GitHub Actions, Jira)
3 Days: Sprint 5 - End-to-End Regression & Defect Retesting
2 Days: Final Test Plan Consolidation & Final Project Submission

14.2 Testing Schedule Table

The following master schedule outlines all testing activities, dependencies, assigned owners, deliverables, and fixed submission milestones.

Phase / Activity	Description & Objective	Duration	Owner / Role	Dependencies	Expected Deliverables	Milestone / Acceptance Criteria
Phase 1: Environment Setup & Requirements Analysis	Clone repo (Hospital-Mangement-System), configure local environment, verify Java/JS dependencies, analyze modules.	2 Days	Ahmed Hussein (Lead) & All Members	GitHub Repo Access, Local Database	Configured Local App (localhost), Functional Requirements Matrix	App runs locally without errors; test environment baseline verified.
Phase 2: Manual Testing & Test Case Design	Design and execute comprehensive manual test cases across all modules (Patient, Doctor, Admin, Billing, Appointments).	10 Days	All Members (Lead: Shaza & Esraa)	Requirements Matrix, Functional App	Manual Test Suites, Execution Logs, Bug Reports in Jira	100% core user workflows covered; all major defects logged.
Phase 3: API Testing (Postman & Newman)	Identify, document, and test backend API endpoints for functional validation, status codes, and payload assertions.	7 Days	All Members (Lead: Ahmed & Alaa)	API Endpoints, Postman	Postman Collections, Newman CLI Execution Reports	All endpoints verified; collection exported to repository.
Phase 4: UI Test Automation (POM & BDD)	Build automated UI test framework using Selenium WebDriver, Page Object Model (POM), and Cucumber BDD scenarios.	14 Days	All Members (Lead: Ahmed & Toqa)	Approved Manual Test Cases, Locators	Automated Test Scripts, BDD Feature Files, HTML Test Reports	Automated regression suite passes locally on Chrome.

Phase / Activity	Description & Objective	Duration	Owner / Role	Dependencies	Expected Deliverables	Milestone / Acceptance Criteria
Phase 5: Performance & Load Testing	Design and execute performance test plans using Apache JMeter to analyze system behavior under load.	7 Days	All Members (Lead: Madeha & Esraa)	Stable Application Build, User Scenarios	JMeter Test Plans (.jmx), Performance Analysis Summary Report	Response time, throughput, and resource utilization logged under simulated load.
Phase 6: CI/CD Pipeline & Tool Integration	Integrate test suites into GitHub Actions CI/CD pipeline triggered on push/pull request; track bugs via Jira.	3 Days	All Members (Lead: Ahmed & Alaa)	GitHub Repository, GitHub Actions	GitHub Actions workflow files, Automated Pipeline Trigger Log, Jira Traceability Matrix	Pipeline automatically runs API & UI suites on push/pull request.
Phase 7: System Regression & Retesting	Execute full regression suite, perform defect retesting, and verify fixed bugs across all application modules.	3 Days	All Members	Resolved Jira Bugs, Fixed Codebase	Retest Execution Logs, Updated Defect Metrics Report	All critical and major defects resolved and verified closed.
Phase 8: Document Integration & Final Submission	Consolidate individual Test Plan sections, resolve inconsistencies, format against PDF template, submit final project.	2 Days	Ahmed Hussein (Lead) & All Section Owners	Complete Section Drafts from All Members	Final Integrated Test Plan PDF, Final Graduation Project Package	Document matches PDF standard; approved by instructor for final submission.

15. Resource Planning

15.1 Roles and Responsibilities Matrix (RACI)

To balance hands-on learning with organizational efficiency, primary leadership is assigned for specific sections, while execution responsibility across all testing types is shared equally among all 6 team members.

Team Member	Project Role	Test Plan Section Ownership	Primary Technical Oversight	Dedicated Weekly Effort
Ahmed Hussein	Team Leader / Senior QA Specialist	Schedule, Milestones, Resource Planning, Document Integration & Final Review	Test Strategy, CI/CD Integration, Automation Architecture	8–10 Hrs/Week
Shaza Rabee	QA Engineer / Business Analyst	Product Analysis, System Description, In-Scope & Out-of-Scope Boundaries	Scope Definition, Manual Test Suite Design	5–8 Hrs/Week

Team Member	Project Role	Test Plan Section Ownership	Primary Technical Oversight	Dedicated Weekly Effort
Esraa Fathy	QA Engineer / Test Strategist	SMART Objectives, Quality Metrics, Test Approach, Entry/Exit Criteria	Methodology, Entry/Exit & Suspension Gates	5–8 Hrs/Week
Alaa Shehata	QA Engineer / Infrastructure Lead	Test Environment Setup, Hardware/Software Specs, System Dependencies	Local Hosting (localhost), DB Setup, Tooling Config	5–8 Hrs/Week
Madeha Mahmoud	QA Engineer / Deliverables Lead	Test Deliverables, RACI Matrix, Communication Protocols, Status Reporting	Artifact Quality, Deliverable Tracking, Status Logs	5–8 Hrs/Week
Toqa Adel	QA Engineer / Risk Analyst	Risk Management, Technical/Operational Bottlenecks, Contingency Plans	Risk Identification, Contingency Buffer Management	5–8 Hrs/Week

15.2 Software, Hardware, and Technical Tools

The technical infrastructure and toolchain selected for the Hospital Management System testing effort are detailed below.

Resource Category	Tool / Infrastructure	Purpose & Application
Operating System	Windows 10/11 / Linux (Ubuntu)	Local development and test execution machines.
Web Browser	Google Chrome (Latest Stable)	Primary target browser for manual testing and UI automation.
Application Tech Stack	Java, JavaScript (65.6%), CSS (23.3%), HTML	Source codebase components hosted on GitHub repository.
Database & Web Server	PostgreSQL 16 / Local App Server	Local backend infrastructure: backend API at http://localhost:8080 and PostgreSQL 16 at localhost:5432 (local run) or localhost:5433 (Docker Compose).
API Testing Tools	Postman & Newman CLI	API endpoint verification, assertion scripting, and CLI automated execution.
UI Automation Framework	Selenium WebDriver (Java 17) + Page Object Model (POM) + Cucumber BDD	Page Object Model (POM) automated functional test scripts with Gherkin BDD syntax.
Performance Testing Tool	Apache JMeter	Load, stress, and response time testing for system endpoints.
CI/CD & Version Control	Git, GitHub, GitHub Actions	Distributed version control, repository tracking, and automated build pipeline execution.
Test & Bug Management	Jira Software + Xray (primary); Microsoft Excel / Google Sheets (alternative/backup)	Defect logging, test case management, task assignment, sprint tracking, and traceability mapping. Excel / Google Sheets are kept as a backup and for supplementary sheets.

15.3 Time Allocation and Workload Model

- Total Team Size: 6 Members
- Individual Availability: 5 to 10 hours per week per member (Minimum total weekly capacity: 30–60 hours)
- Execution Allocation:
- Manual Testing & Test Case Design: 30% of total effort
- API & UI Test Automation Scripting: 35% of total effort
- Performance & Load Testing: 15% of total effort
- CI/CD Configuration & Tooling: 10% of total effort
- Documentation, Review, and Meetings: 10% of total effort

16. Communication and Reporting

Effective communication is important to ensure that all QA team members are informed about testing progress, identified defects, and any challenges during the testing process.

The QA team will use **Zoom** for team meetings and discussions, **Jira Software with Xray** to manage bugs and test cases, **GitHub** for code and CI/CD, and **Microsoft Excel or Google Sheets** as an alternative/backup and for supplementary testing documentation.

16.1 Communication Plan

Communication Activity	Purpose	Frequency	Participants
Daily Progress Updates	Discuss completed tasks, current testing progress, and upcoming activities.	Daily	QA Team
Bug Review	Review identified defects, their severity, priority, and status.	As needed	QA Team
Automation Testing Updates	Discuss automated test script development, execution results, and issues.	Daily during automation activities	QA Team
Performance Testing Updates	Share JMeter results, threshold breaches, and bottlenecks found.	During performance testing activities	QA Team
API Testing Updates	Share testing results, access limitations, and identified API issues.	During related testing activities	QA Team
Test Execution Review	Review passed, failed, blocked, and pending test cases.	Daily	QA Team
Final Testing Review	Review the final test results, completed deliverables, and project readiness.	At the end of testing	QA Team

16.2 Communication Tools

Zoom

- Conduct team meetings.
- Discuss testing progress and assigned tasks.
- Review reported defects and testing challenges.
- Coordinate Manual, Automation, API, and Performance Testing activities.

Jira Software + Xray (Primary Tool)

- Log, assign, and track bugs from discovery through retest and closure.

- Manage test cases and link them to defects for traceability.
- Track sprint tasks and testing progress.
- Generate defect reports and metrics for the Test Summary Report.

Microsoft Excel / Google Sheets (Backup and Supplementary Sheets)

- Keep backup copies of test cases exported from Jira.
- Keep a backup list of reported bugs exported from Jira.
- Update test execution results.
- Share daily testing progress.
- Prepare final testing reports.

16.3 Reporting Frequency

The QA team will update the testing progress **daily**.

Daily updates will include:

- Completed and pending testing activities.
- Number of passed, failed, and blocked test cases.
- Newly identified defects.
- Automation, API, and Performance Testing progress, where applicable.
- Issues or blockers affecting testing activities.
- Planned tasks for the next testing session.

16.4 Final Communication and Reporting

At the end of the testing process, the QA team will conduct a final meeting through Zoom to review the testing activities and results.

A **Test Summary Report** will be prepared to summarize the completed tests, identified defects, testing limitations, and overall quality status of the Hospital Management System (HMS).

All final deliverables will be organized and prepared for graduation project submission.

17. Document Integration and Final Review

17.1 Document Integration Strategy

As the Team Leader, Ahmed Hussein oversees the consolidation of all six individual Test Plan components into a unified deliverable. To ensure strict alignment with the instructor's requirements, the document integration workflow follows three main review steps:

Step 1: Structural Audit – Verify all 8 core PDF sections are present and follow a consistent header hierarchy.

Step 2: Cross-Sectional Consistency Audit – Cross-check Scope ↔️ Approach ↔️ Tools ↔️ Schedule ↔️ Risks.

Step 3: Quality Gate Verification & Final Sign-Off – Run the Final Submission Checklist prior to submission on Dec 10, 2026.

17.2 Pre-Submission Integration Checklist

Audit Category	Checkpoint Description	Status	Responsible Reviewer
Structural Alignment	Are all 8 mandatory PDF sections included (Objectives, Approach, Scope, Deliverables, Dependencies, Environment, Risk Management, Schedule)?	Completed	Ahmed Hussein
Terminology Consistency	Are tool names (Postman, Newman, JMeter, GitHub Actions, Jira, Xray, Selenium WebDriver, Page Object Model (POM), Cucumber BDD) and stack terms spelled consistently throughout?	Completed	Ahmed Hussein
Cross-Sectional Dates	Do the dates in the Schedule match the milestone dates mentioned in Scope and Risk Management?	Completed	Ahmed Hussein & Toqa Adel
Resource Traceability	Are all six team members explicitly listed with clear RACI role assignments?	Completed	Ahmed Hussein & Madeha Mahmoud
Scope Boundary Audit	Does the test schedule explicitly cover every module listed under In-Scope (Patient, Doctor, Admin, Billing)?	Completed	Ahmed Hussein & Shaza Rabee
Criteria Alignment	Are the Entry, Exit, and Suspension criteria in the Approach section aligned with the Regression Phase timeline?	Completed	Ahmed Hussein & Esraa Fathy
Formatting & Layout	Does the document layout mirror the instructor's reference PDF template?	Completed	Ahmed Hussein

17.3 Final Quality Review Checklist (Instructor PDF Compliance)

- SMART Criteria Applied: Objectives are Specific, Measurable, Attainable, Relevant, and Time-Bound.
- Distinction Maintained: Test Plan specifics (project-tailored) are clearly separated from high-level Test Strategy principles.
- Dependencies Mapped: Local host dependencies, database requirements, and GitHub code states are documented.
- Risk & Contingencies Defined: Technical setup bottlenecks and mitigation strategies are mapped directly to schedule buffers.
- Artifact Deliverables Defined: Test cases, Jira defect logs, Postman collections, JMeter execution reports, and GitHub Actions pipelines are cataloged.